

Chi Avançado, Validação e Middleware

DIM0547 – Sprint 1

Prof. Fernando · UFRN · 2026.1

Onde estamos

- **No vídeo anterior:** Chi básico — `chi.NewRouter()`, `r.Get/Post`, `chi.URLParam`, refatoração do ex01
- **Neste vídeo:** Chi avançado — grupos de rotas, middleware, validação, erros padronizados

Depois deste vídeo vocês têm tudo para:

-  Fazer os 4 exercícios da **Lista 3**
-  Entregar a **Sprint 1** do projeto

PARTE 1 – Organizando rotas

0 problema: tudo em main.go

```
func NewRouter() http.Handler {  
    r := chi.NewRouter()  
    r.Get("/api/v1/contacts", listContacts)  
    r.Post("/api/v1/contacts", createContact)  
    r.Get("/api/v1/contacts/{id}", getContact)  
    r.Delete("/api/v1/contacts/{id}", deleteContact)  
    r.Get("/api/v1/orders", listOrders)  
    r.Post("/api/v1/orders", createOrder)  
    r.Get("/api/v1/orders/{id}", getOrder)  
    r.Get("/api/v1/products", listProducts)  
    // ... mais 20 rotas ...  
    return r  
}
```

Com 30 endpoints, esse arquivo tem **500 linhas**. Impossível de manter.

Grupos de rotas com r.Route

```
r := chi.NewRouter()

r.Route("/api/v1", func(r chi.Router) {
    r.Get("/contacts", listContacts)
    r.Post("/contacts", createContact)
    r.Get("/contacts/{id}", getContact)
    r.Delete("/contacts/{id}", deleteContact)
})

r.Route("/api/v2", func(r chi.Router) {
    r.Get("/contacts", listContactsV2) // formato diferente
})

r.Get("/health", healthCheck) // fora dos grupos
```

Prefixo declarado uma vez, escopo claro. Versionamento de API com **zero repetição**.

Isso é o **ex02** da Lista 3.

Subrouters com r.Mount

Cada recurso em seu próprio pacote:

```
// internal/handler/contacts.go
func ContactsRouter() chi.Router {
    r := chi.NewRouter()
    r.Get("/", listContacts)
    r.Post("/", createContact)
    r.Get("/{id}", getContact)
    r.Delete("/{id}", deleteContact)
    return r
}
```

No `main.go` — só composição:

```
r := chi.NewRouter()
r.Mount("/api/v1/contacts", handler.ContactsRouter())
r.Mount("/api/v1/orders", handler.OrdersRouter())
```

Cada módulo cuida das suas rotas. O main só compõe.

r.Route vs r.Mount: quando usar cada

r.Route — rotas inline

```
r.Route("/api/v1", func(r chi.Router) {  
    r.Get("/contacts", list)  
    r.Post("/contacts", create)  
})
```

Bom para: **agrupar logicamente** dentro do mesmo arquivo

r.Mount — router externo

```
r.Mount("/api/v1/contacts",  
    handler.ContactsRouter())
```

Bom para: **separar em pacotes** diferentes

Podem ser combinados:

```
r.Route("/api/v1", func(r chi.Router) {  
    r.Mount("/contacts", handler.ContactsRouter())  
    r.Mount("/orders", handler.OrdersRouter())  
})
```

PARTE 2 – Middleware

Middleware: o padrão universal de Go

```
func MeuMiddleware(next http.Handler) http.Handler {  
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {  
        // ANTES do handler  
        log.Println("request chegou:", r.Method, r.URL.Path)  
  
        next.ServeHTTP(w, r) // chama o próximo  
  
        // DEPOIS do handler  
        log.Println("response enviada")  
    })  
}
```

O **padrão**: recebe `http.Handler`, retorna `http.Handler`.

Esse padrão é da **stdlib de Go**, não do Chi. Funciona em qualquer lugar.
É o **ex03** da Lista 3.

Middleware global vs por grupo

```
r := chi.NewRouter()

// — Global: todas as rotas —
r.Use(middleware.Logger)
r.Use(middleware.Recoverer)

// — Sem auth —
r.Get("/health", healthHandler)
r.Get("/ping", pingHandler)

// — Com auth (só /api) —
r.Route("/api", func(r chi.Router) {
    r.Use(authMiddleware) // ← só aqui dentro
    r.Get("/contacts", listContacts)
    r.Get("/orders", listOrders)
})
```

`/health` → Logger + Recoverer

`/api/contacts` → Logger + Recoverer + **authMiddleware**

Middleware stack: ordem importa

```
r.Use(middleware.RequestID) // 1º – gera ID
r.Use(middleware.Logger)    // 2º – loga (já com ID)
r.Use(middleware.Recoverer) // 3º – captura panic
```

```
Request  → RequestID → Logger → Recoverer → Handler
Response ← RequestID ← Logger ← Recoverer ← Handler
```

RequestID antes de Logger — senão o log não tem o ID.

Recoverer por último — captura panics de tudo que vem antes.

Middleware embutidos do Chi

`github.com/go-chi/chi/v5/middleware`

Middleware	O que faz
Logger	Log de método, path, status, duração
Recoverer	Captura panics → 500 em vez de crash
RequestID	Header <code>X-Request-Id</code> único por request
RealIP	IP real atrás de proxy/load balancer
Throttle(n)	Limita N requests simultâneos
Timeout(d)	Cancela request após duração

```
import chiware "github.com/go-chi/chi/v5/middleware"
```

```
r.Use(chiware.Logger)  
r.Use(chiware.Recoverer)
```

Na Sprint 1 vocês precisam de **pelo menos 1 middleware** (logging OU recovery).

PARTE 3 – Validação

0 problema: validação manual

```
func createUser(w http.ResponseWriter, r *http.Request) {  
    var req CreateUserRequest  
    json.NewDecoder(r.Body).Decode(&req)  
  
    if req.Name == "" {  
        http.Error(w, "name required", 400)  
        return  
    }  
    if len(req.Name) < 2 {  
        http.Error(w, "name too short", 400)  
        return  
    }  
    if req.Email == "" {  
        http.Error(w, "email required", 400)  
        return  
    }  
    // ... 20 linhas de if/else ...  
}
```

Repetitivo, propenso a erro, inconsistente. **Não escala.**

Validação declarativa com go-playground/validator

```
go get github.com/go-playground/validator/v10
```

```
type CreateUserRequest struct {  
    Name      string `json:"name"      validate:"required,min=2,max=100"`  
    Email     string `json:"email"     validate:"required,email"`  
    Age       int    `json:"age"       validate:"required,min=18,max=120"`  
    Password  string `json:"password"  validate:"required,min=8"`  
}
```

Tags `validate` nos campos da struct. Sem `if/else`.

Tag	Significado
<code>required</code>	Campo obrigatório (não pode ser zero value)
<code>min=2</code>	Mínimo 2 (caracteres p/ string, valor p/ int)
<code>max=100</code>	Máximo 100
<code>email</code>	Formato de email válido

Usando o validator no handler

```
import "github.com/go-playground/validator/v10"

var validate = validator.New()

func createUser(w http.ResponseWriter, r *http.Request) {
    var req CreateUserRequest

    // 1. Decodificar JSON
    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
        // JSON inválido → 400
        writeProblem(w, ProblemDetails{
            Status: 400, Title: "Bad Request", Detail: "invalid json",
        })
        return
    }

    // 2. Validar
    if err := validate.Struct(&req); err != nil {
        // Validação falhou → 422 com lista de erros
        writeProblem(w, ProblemDetails{
            Status: 422, Title: "Validation Error",
            Detail: "Um ou mais campos são inválidos",
            Errors: extractFieldErrors(err),
        })
        return
    }

    // 3. Campos válidos – criar usuário
}
```


Tags comuns do validator

Tag	Exemplo	Valida
<code>required</code>	<code>validate:"required"</code>	Campo não pode ser zero value
<code>email</code>	<code>validate:"email"</code>	Formato de email
<code>min</code>	<code>validate:"min=8"</code>	Mín. 8 chars (string) ou valor 8 (int)
<code>max</code>	<code>validate:"max=100"</code>	Máx. 100
<code>len</code>	<code>validate:"len=11"</code>	Tamanho exato
<code>oneof</code>	<code>validate:"oneof=admin user"</code>	Um dos valores listados
<code>url</code>	<code>validate:"url"</code>	URL válida
<code>gte</code>	<code>validate:"gte=0"</code>	≥ 0 (para números)
<code>e164</code>	<code>validate:"e164"</code>	Telefone internacional (+55...)

Combinar com vírgula: `validate:"required,email"`

Referência completa: github.com/go-playground/validator

PARTE 4 – Erros padronizados

0 problema: cada API inventa seu formato de erro

```
{"error": "not found"}  
{"message": "invalid", "code": 422}  
{"errors": ["name required", "email invalid"]}  
{"success": false, "reason": "unauthorized"}
```

Cada API retorna erros de um jeito diferente. O cliente não sabe o que esperar.

Solução: RFC 7807 — Problem Details for HTTP APIs

Um formato padronizado que qualquer cliente sabe ler.

Problem Details (RFC 7807 / 9457)

```
{
  "type": "https://example.com/errors/validation",
  "title": "Validation Error",
  "status": 422,
  "detail": "Um ou mais campos são inválidos",
  "errors": [
    {"field": "email", "message": "must be a valid email"},
    {"field": "age", "message": "must be at least 18"}
  ]
}
```

Content-Type: application/problem+json

Campo	Obrigatório	Significado
type	sim	URI que identifica o tipo de erro
title	sim	Título legível do erro
status	sim	Código HTTP
detail	não	Descrição específica desta ocorrência
errors	não	Lista de erros por campo (extensão)

Implementando Problem Details em Go

```
type FieldError struct {
    Field    string `json:"field"`
    Message  string `json:"message"`
}

type ProblemDetails struct {
    Type      string    `json:"type"`
    Title     string    `json:"title"`
    Status    int       `json:"status"`
    Detail    string    `json:"detail"`
    Errors    []FieldError `json:"errors,omitempty"`
}

func writeProblem(w http.ResponseWriter, p ProblemDetails) {
    w.Header().Set("Content-Type", "application/problem+json")
    w.WriteHeader(p.Status)
    json.NewEncoder(w).Encode(p)
}
```

`omitempty` em `Errors` : se não houver erros de campo, o array nem aparece no JSON.

Isso é o **ex04** da Lista 3.

Extraindo erros do validator para FieldError

```
func extractFieldErrors(err error) []FieldError {
    var ve validator.ValidationErrors
    if !errors.As(err, &ve) {
        return nil
    }
    out := make([]FieldError, 0, len(ve))
    for _, fe := range ve {
        out = append(out, FieldError{
            Field: strings.ToLower(fe.Field()),
            Message: messageFor(fe),
        })
    }
    return out
}

func messageFor(fe validator.FieldError) string {
    switch fe.Tag() {
    case "required": return "is required"
    case "email":    return "must be a valid email"
    case "min":      return fmt.Sprintf("must be at least %s", fe.Param())
    case "max":      return fmt.Sprintf("must be at most %s", fe.Param())
    default:         return fmt.Sprintf("failed: %s", fe.Tag())
    }
}
```

Padrão completo: decode → validate → respond

```
func (a *app) createUser(w http.ResponseWriter, r *http.Request) {
    var req CreateUserRequest

    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
        writeProblem(w, ProblemDetails{
            Type: "https://example.com/errors/bad-request",
            Title: "Bad Request", Status: 400,
            Detail: "invalid json body",
        })
        return
    }

    if err := a.validate.Struct(&req); err != nil {
        writeProblem(w, ProblemDetails{
            Type: "https://example.com/errors/validation",
            Title: "Validation Error", Status: 422,
            Detail: "Um ou mais campos são inválidos",
            Errors: extractFieldErrors(err),
        })
        return
    }

    user := User{ID: nextID(), Name: req.Name, Email: req.Email, Age: req.Age}
    a.users[user.ID] = user
    writeJSON(w, http.StatusCreated, user)
    // ⚠ Senha NUNCA aparece na resposta (User não tem campo Password)
}
```

PARTE 5 – Juntando tudo para a Sprint 1

0 que a Sprint 1 pede

Requisito	Como resolver
API usando Chi como router	<code>chi.NewRouter()</code> + <code>r.Get/Post/Delete</code>
Pelo menos 4 endpoints	CRUD de 1 recurso já dá 4
Pelo menos 1 middleware	<code>chiware.Logger</code> ou <code>chiware.Recoverer</code>
Testes automatizados verdes	<code>httptest</code> + <code>go test ./...</code>
GitHub Actions passando	CI em <code>.github/workflows/ci.yml</code>
Vídeo 8 min	Demonstrar API + tour pelo código

Avaliação: funcionalidade (40%) + qualidade de código e testes (35%) + vídeo (25%)

Estrutura de projeto recomendada

```
meu-projeto/
├── cmd/api/
│   └── main.go                # Setup + ListenAndServe
├── internal/
│   ├── handler/
│   │   ├── contacts.go       # Router + handlers
│   │   └── contacts_test.go
│   ├── middleware/
│   │   ├── logging.go        # (ou use chiware.Logger)
│   │   └── recovery.go
│   └── model/
│       └── contact.go
├── go.mod
└── .github/workflows/ci.yml
```

Não precisa ser exatamente assim. Mas **todo código em** `main.go` não é aceitável para a Sprint 1.

Lista 3 → Sprint 1: conexão direta

Exercício	O que pratica	Como aparece na Sprint 1
ex01 — Migrar para Chi	<code>chi.NewRouter</code> , <code>chi.URLParam</code>	Router do projeto
ex02 — Grupos e subrouters	<code>r.Route</code> , <code>r.Mount</code>	Organização de rotas
ex03 — Middleware	<code>r.Use</code> , escopo por grupo	Logging, recovery, auth
ex04 — Validação + RFC 7807	<code>validator</code> , <code>ProblemDetails</code>	Validação dos endpoints

Quem fizer a Lista 3 bem feita já tem **80% do código** da Sprint 1.

Resumo do que vocês sabem agora

Conceito	Slide	Lista 3
Grupos de rotas (<code>r.Route</code>)	✓	ex02
Subrouters (<code>r.Mount</code>)	✓	ex02
Middleware por grupo (<code>r.Use</code>)	✓	ex03
Middleware customizado	✓	ex03
<code>go-playground/validator</code>	✓	ex04
Tags <code>validate:"required,email"</code>	✓	ex04
Problem Details (RFC 7807)	✓	ex04
Extrair erros do validator	✓	ex04
CI com GitHub Actions	✓	Sprint 1
Estrutura de projeto	✓	Sprint 1

Próximo vídeo: persistência com PostgreSQL e sqlc.